

LEAN: Light and Efficient Audio Classification Network

Shwetank Choudhary
Samsung R&D Institute

Bangalore, India
sj.choudhary@samsung.com

CR Karthik
Samsung R&D Institute

Bangalore, India
cr.karthik@samsung.com

Punuru Sri Lakshmi
Samsung R&D Institute

Bangalore, India
srilakshmi.p@samsung.com

Sumit Kumar
Samsung R&D Institute

Bangalore, India
sumit.kr@samsung.com

Abstract—Over the past few years, audio classification task on large-scale dataset such as AudioSet has been an important research area. Several deeper Convolution-based Neural networks have shown compelling performance notably Vggish, YAMNet, and Pretrained Audio Neural Network (PANN). These models are available as pretrained architecture for transfer learning as well as specific audio task adoption. In this paper, we propose a lightweight on-device deep learning-based model for audio classification, LEAN. LEAN consists of a raw waveform-based temporal feature extractor called as Wave Encoder and logmel-based Pretrained YAMNet. We show that using a combination of trainable wave encoder, Pretrained YAMNet along with cross attention-based temporal realignment, results in competitive performance on downstream audio classification tasks with lesser memory footprints and hence making it suitable for resource constraints devices such as mobile, edge devices, etc. Our proposed system achieves on-device mean average precision(mAP) of .445 with a memory footprint of a mere 4.5MB on the FSD50K dataset which is an improvement of 22% over baseline on-device mAP on same dataset.

Index Terms—audio classification, cross attention, sound classification

I. INTRODUCTION & RELATED WORK

In recent years, along with speech recognition, identifying sound types using raw signals has become an emerging research area. Typically, audio pattern recognition is the umbrella word that consists of many tasks such as audio tagging [1], acoustic scene classification [2], audio classification [3], and sound event detection. Identifying the type of sound from real-time signal has several use cases such as hearing disability removal, accessibility, intelligent smartphones, improving gaming experience, etc. Deep learning-based methods are de facto advanced solutions [4], [5], [6], [7], [8] to detect a sound event in the digital signal. The task of identifying the sound typically requires a sufficiently large dataset especially when the problem demands multi-class multi-label classification. Several datasets have been released in previous years namely Kaggle Free sound dataset [14], UrbanSound8K [12], and ESC50 [13]. One of the greatest breakthroughs in the audio classification task is the release of a large-scale audio dataset called AudioSet [10] which is a publicly available dataset having 500 hours of clips corresponding to 527 classes. Dataset is based on Youtube videos and clips. However, some of the clips have been deleted and hence putting a limitation

on open usage. In the year 2020, one effort to release a large-scale open dataset has been done by the research community by releasing Free Sound Dataset (FSD50K) [9] with raw files available under an open license. Our present work is based on this FSD50K dataset. Identifying sound types for the multi-class multi-label dataset is a complex task. Robust models for such types of tasks require deep and complex networks which are capable of capturing sound patterns. Taking inspiration from the vision-based task, CNNs [15], [16] have been extensively used by researchers to detect sound patterns [17], [18], emotion [6], and language detection [11], [19]. Such models typically take fixed input ranging from 1 second [7], [8] to 10 seconds [11], [19] and produce fixed size embedding [7], [8] which is used for the classification task. Several deep and complex models have also been designed [4], [7] which have achieved the state of the art performance on AudioSet. In a recent breakthrough, attention-based networks proposed by Bahdanau's & Loung's [20], [22] and multi-attention-based transformers [21], [5] have been widely adopted for vision, and audio tasks and have shown significant performance gain. One such example is the Audio transformer [5] on FSD50K which achieved a mean average precision score (mAP) of .537. It consists of a stack of transformer blocks where each block is followed by average pooling except the last block. Similarly, transformer-based Patchout faSt Spectrogram Transformer (PaSST) [31] tries to overcome the computational complexity of transformers by introducing a method called Patchout which is the current state of the art on FSD50K with .653 mAP. However, despite performing so well, transformers are costly both in memory and computation level and difficult to port on-device. For example in PaSST, various models discussed have parameters varying from 50M to 80M which translates to very large model size. PSLA [32] proposes a collection of training techniques such as ImageNet pretraining, balanced sampling, label enhancement, model aggregation, etc. to boost the model accuracy and achieve the best mAP of .5671 on FSD50K with ensemble models. PSLA uses several EfficientNet (B0, B2 with attention) based architectures for ensembling and their base model called EfficientNet-B0 with single-headed attention has 5.36M model parameters on the AudioSet dataset. BYOL-A [33] pretrains representations of the input sound invariant to audio data augmentations, which makes the learned representations robust to the perturbations of

sounds and achieves the best mAP of .448 on FSD50K. Some shortcomings with the above-discussed work are increased model complexity, larger training time coupled with large model size which leads to increased computation and memory cost and thus makes them unsuitable for resource constraints environments such as mobile devices. Our work tries to reflect on these issues as it is a simple reproducible pipeline that uses Pretrained Yamnet and trainable Wave Encoder with just 4.58M model parameters and quantized on-device model size is 4.5MB and hence suitable for on-device deployment. Also, our proposed model requires just 1 second of the audio frame to make predictions whereas several of these works [31], [32] take input frame duration ranging from 10 to 30 seconds and thus make our system computationally efficient due to reduced pre-processing time for creating spectrograms. To the best of our knowledge, Temporal Knowledge distillation for on-Device audio classification [35] only works on FSD50K which focuses on developing a lightweight system for on-device deployment. This [35] work adopts Teacher-Student based architecture for knowledge transfer and utilizes a heavy transformer-based model as a teacher and on-device models of various architectures as a student. Since this [35] work provides on-device model performance, we baseline this work for comparing our model results. Among the several baseline models discussed in the work, Att-RNN achieves the best on-device mAP of .3471 whereas our model achieves an mAP of .445 on the same dataset which is a 22% improvement over the baseline model.

Mostly frequency domain features such as log-mel and Short-time Fourier transform (STFT) based spectrogram have been common choices [16], [17], [18] as input for an audio classification task. However, PANN [4] model on Audioset has demonstrated that time-domain features can act as a supplement to Frequency domain features in improving the model performance. We also get the inspiration from PANN to consider both time and frequency domain features as input to our model. In this work, we propose a novel lightweight network called as Light and Efficient Audio Classification Network(LEAN) for on-device audio classification. LEAN consists of waveform based temporal feature extractor called as Wave Encoder and logmel-based Pretrained YAMNet as a feature extractor and cross attention based temporal feature realignment scheme. Our contributions, in this study, can be summarized in the following way:

- We propose a lightweight on-device novel network called as LEAN which takes temporal and frequency features as input.
- We introduce temporal feature realignment scheme using pre-trained embeddings through cross attention which leads to improved performance with slight increase in training parameters.
- We show that despite being lightweight in memory and smaller frame input of 1 second, we achieve competitive performance with .4677 as the best mAP on GPU and .445 on-device mAP with just 4.5MB model

size(quantized) on FSD50K dataset.

- We provide class level performance analysis and impact by our pipeline.

II. DATASET

For our system evaluation, we use Free Sound Dataset (FSD50K) [9]. It is an open dataset containing over 51,000 audio clips for sound events and has 200 classes drawn from the AudioSet Ontology [25]. The total duration of all clips for this dataset is 108 hours. Sound clips in the dataset are weakly labeled i.e. labeling of classes is done at clip level instead of frame level. Clips are multi-labeled and range from 0.3-30seconds in duration. Audio files consist of sounds from humans, sounds of things, animals, natural sounds, musical instruments, and more. We adopt FSD50K dataset as it is freely available under the creative commons license and popularly available for benchmarking Sound Event Detection models. FSD50K dataset is divided into three parts namely training, validation, and evaluation. We use training and validation split for training, fine-tuning the model and evaluation split for showcasing our results. For benchmarking our model performance, we use the same metrics as mentioned in the baseline system [9] such as mAP(Mean Average Precision). Mean average precision is an approximation of the area under the class's precision-recall curve, which is more informative of performance when dealing with imbalanced datasets such as AudioSet and FSD50K in comparison to the average area under the curve of the receiver operating characteristic curve [36]. In addition, we also calculate mAUC-PR (Mean Area Under the Curve for Precision-Recall) and sensitivity index (d-prime) as an additional metric. We also refer to the unofficial implementation of FSD50K baseline systems [29] for producing our results.

III. METHODOLOGY

This section describes methods applied for audio classification. Our end-to-end proposed architecture is shown in Fig. 1. Here, we begin by introducing preprocessing steps that were applied for audio feature generation. We then, describe our proposed Wave Encoder along with a pre-trained YAMNet-based model. We then discuss our proposed system for cross-attention-based feature realignment.

A. Dataset Preprocessing

We downsample all audio clips to 16k Hz using the Sound library [26]. We consider a 1-second audio patch as input for all our training and testing. Labels for each 1-second patch are the same as that of the complete audio clip. We repeat the data for those clips whose duration was less than 1 second to get a fixed 1-second duration of the sample required by our model.

Our model consists of a two-channel network and hence takes two different inputs: 1. Raw waveform 2. Log-mel Spectrogram. Raw waveform for the 1-second patch is directly considered as input to Wave Encoder. For creating the log-mel spectrogram, we use the same setup and parameters as used

in YAMNet. YAMNet takes a log-mel spectrogram having the shape as (96,64) which is corresponding to 960ms audio data. In each patch, 64 being mel bins and 96 being time dimension. At broader level, parameters are : Window length = 25ms, hop length = 10ms, Number of Mel Bands = 64, Mel Frequency range = [125 Hz, 7500 Hz].

B. Pretrained YAMNet as Feature Extractor

One of the channels (we call it as the right channel) in our proposed model is CNN based feature extractor which captures spatial features of the given audio frame. Among the available large-scale audio dataset-based pretrained models for transfer learning, we choose YAMNet [8] over Vggish [7] and PANN's [4]. YAMNet is mobilenet based architecture with the adoption of depth and pointwise convolution. It is a lightweight pretrained model with nearly 3.7 million weights and trained on the AudioSet dataset and achieved .306 mAP on the evaluation set. Although both Vggish and PANN models are better in terms of the performance compared to YAMNet but they are heavy networks both in memory and computation and hence not suitable for resource constraints environments like mobile phones and edge devices.

In our various experiments discussed below, we add a dense layer of 256 units (called as projection layer) after 1024 embedding vector of YAMNet as shown in Fig. 1. The reason is to make adjustments concerning Wave Encoder output and to reduce joint embedding size.

C. Proposed Wave Encoder as Temporal Feature Extractor

Taking inspiration from the PANN model, we propose to use raw waveform input to capture time-domain features. In the PANN model, temporal features are captured by the Wavegram model which is a 1D Convolutional-based network. In contrast, the proposed Wave Encoder is a Bidirectional LSTM-based network. Wave Encoder takes raw waveform input and outputs learned temporal features. We call this part of the network as a left channel. It contains two Bi-directional LSTM layers each of 128 units. Since LSTM takes time sequence input, we first transform raw waveform into time sequence data by reshaping it. Reshape layer converts waveform into 2D time sequence data by splitting into patches using a non-overlapping window of 25 milliseconds. For 1 second frame with a sampling rate of 16K, reshape layer outputs a 2D vector as (40,400).

D. Proposed Joint Model using Time and Frequency Domain features

We propose a joint model using Wave Encoder as the left channel and pretrained YAMNet with a projection layer as the right channel. Projection layer outputs embedding $E_{yam} = e_1, e_2, \dots, e_n$. Wave Encoder encodes raw waveform data into a context vector C_t . Outputs from both the channels are combined using the following equation.

$$E_{combined} = concatenate(E_{yam}, C_t) \quad (1)$$

If Y_i is true multi label vector for i^{th} audio sample and $\hat{Y}_i$ is predicted score vector, then our objective function can be defined as

$$\hat{Y}_i = sigmoid(E_{combined}^T P + b) \quad (2)$$

$$Loss = BCE(Y_i, \hat{Y}_i) \quad (3)$$

Where $E_{combined}$ is a joint embedding vector of $E_{yam} \in R^k$

and $C_t \in R^k, E_{combined} \in R^k, P \in R^{k \times c}, b$ as bias

Here k is vector dimension and c is the number of classes. BCE is the binary cross entropy loss for multi label multi class problem.

During training, E_{yam} is pretrained part and only C_t part is learnt and updated during backpropagation.

TABLE I
BASELINE, SOTA AND PROPOSED SYSTEM PERFORMANCE ON FSD50K DATASET

Model Name	mAP	mAUC	dPrime	Total Param
CRNN [9] (Baseline)	.417	-	2.068	0.96M
VGG-like [9] (Baseline)	.434	-	2.167	0.27M
ResNet-18 [9] (Baseline)	.373	-	1.883	11.3M
DenseNet-121 [9] (Baseline)	.425	-	2.112	12.5M
Audio Transformer [5]	.537	-	-	2.3M
PSLA [32]	0.5671	-	-	13.64M
BYOL-A [33]	0.448	0.896	-	6.3M
PaSST [31] (SOTA)	.653	-	-	50M
Wav2CLIP [34]	0.4308	-	-	11M
TKD [35]	0.3471	-	-	0.2M
Joint Model + Bahdanau Cross Attention (LEAN)	0.4677	0.944	2.251	4.58M

E. Temporal feature realignment using Cross Attention for Joint Model

Instead of the simple concatenation of output from Wave encoder and YAMNet, we propose two realignment schemes using cross attention for getting final embeddings.

- Cross attention as simple affinity score
- Cross attention similar to Bahdanau's type attention with learning weights

a) *Cross attention as simple affinity score* : Inspired by the work "Multimodal Speech Emotion Recognition Using Audio and Text" [6], we adopt similar cross attention for our proposed lightweight system to improve the model performance without a significant increase in model parameters. The said work uses audio embeddings to realign textual embeddings. In a similar fashion, we propose to use pretrained YAMNet embeddings to realign Wave Encoder output using

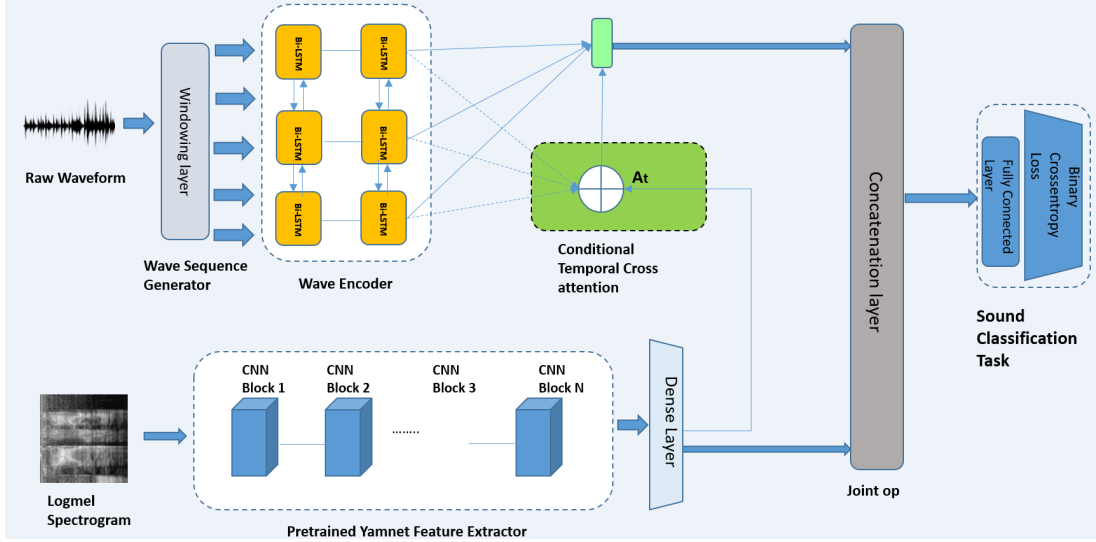


Fig. 1. Our Proposed Model Architecture (LEAN)

cross attention. In this scheme of attention, we use a simple dot product between YAMNet and Wave Encoder output to calculate affinity scores of time sequences from Wave Encoder. Equations (4) to (7) discuss the mathematical formulation of the cross attention scheme. Here, if E_{yam} is output of pretrained YAMNet embeddings after projection layer and h_t is hidden state of t^{th} sequence of Wave Encoder output, then A_t as attention vector is calculated using equation (4). C_{att} in equation (5) is the attentive context vector which is calculated using a weighted sum of T hidden states. C_{att} is then concatenated with E_{yam} to get final joint embeddings. Our objective function remains the same as discussed in equations (2) and (3).

$$A_t = \text{softmax}(\tanh(\text{Dot}(E_{yam}, h_t))) \quad (4)$$

$$C_{att} = \sum A_t h_t \quad (5)$$

$$E_{combined(att)} = \text{concatenate}(E_{yam}, C_{att}) \quad (6)$$

$$\hat{Y}i = \text{sigmoid}(E_{combined(att)}^T P + b) \quad (7)$$

b) Cross attention similar to Bahdanau's type attention with learning weights: We experiment Bahdanau's additive style of attention with learning weights using dense layers. We conceptualize cross attention as Query (Q), Key (K), and Value (V) in a similar fashion as done in the Transformer network [21]. In this scheme, projection layer output embeddings are chosen as Q, and Wave Encoder output is chosen as K, V.

The key difference in this method in comparison to affinity attention is in calculating A_t attention vector which is discussed in equations (8) to (11). The below equations discuss the calculations involved in the cross-attention scheme.

$$Q = (E_{yam}^T W_q + b_q) \quad (8)$$

$$K = (h^T W_k + b_k) \quad (9)$$

$$V = (\tanh(Q + K) W_v + b) \quad (10)$$

$$A_t = \text{softmax}(V) \quad (11)$$

Where $E_{yam} \in \mathbb{R}^{(m)}$, $W_q \in \mathbb{R}^{m \times d}$, $h \in \mathbb{R}^{(t \times m)}$

$W_k \in \mathbb{R}^{m \times d}$, $W_v \in \mathbb{R}_{m \times 1}$

Here d is dense layer units and is chosen as 128 after several experiments.

F. Training and Hyper Parameter Selection

We use Tensorflow-GPU 2.3.0 [27] and Keras 2.6.0 [28] for all our implementation. For all our training, we fix batch size as 64, learning rate as $1e-4$, and loss as binary cross-entropy.

Training is done for 40 epochs with each epoch roughly taking 1.5 hours. We observe that models generally converge well before the last epoch. We select the best model having the highest validation AUC (PR).

IV. RESULTS & DISCUSSION

We use Nvidia GPU GeForce GTX 1080 Ti 11178 MB card for training and testing. All models are trained and tested end to end. Testing is done on the evaluation dataset provided by FSD50K. For testing, a given audio file is split into chunks of 1 second each with an overlap of 50%. For each chunk, the raw waveform and corresponding log-mel (with mean normalization) are calculated and fed to the network. Chunk level class predictions score is calculated and averaged to

TABLE II
CLASS WISE MAP IMPROVEMENT USING PROPOSED MODEL ON 200 CLASSES FOR FSD50K

<i>Baseline& Reference Models</i>		<i>Count of classes which see improvement(mAP) over B by R out of 200 classes</i>			
<i>Baseline Model(B)</i>	<i>Reference Model(R)</i>	<i>Overall Improvement</i>	<i>Better by 5%</i>	<i>Better by 10%</i>	<i>Better by 20%</i>
YAMNet Finetune	Joint Model	140	55	31	13
YAMNet Finetune	Joint Model with Cross Attention	135	53	30	14
Joint Model	Joint Model with Cross Attention	96	27	9	3

get a final class-wise score for the entire file. We observe a better performance with an overlapping split compared to the non-overlapping one. We believe this is mainly due to the availability of more data by the overlapping split.

Below, we discuss all results and the type of attention network we adopt.

A. Fine Tuning YAMNet

As discussed in section III-B, we choose YAMNet as our feature extractor which gives 1024 vectors as latent embedding. We experiment in multiple ways to fine-tune YAMNet to achieve its best performance on FSD50K. For this, we retrain YAMNet end to end with and without pretrained weights initialization. However, we observe that none could outperform the baseline system [9]. We then directly use pretrained embedding with a projection dense layer of 256 and achieved an mAP of 0.4577 outperforming baseline results of VGG-like having mAP of .434.

TABLE III
ABLATION STUDY

<i>Model</i>	<i>mAP</i>	<i>mAUC</i>	<i>dPrime</i>	<i>Total Param</i>
YAMNet (without finetune)	0.446	0.939	2.192	3.42M
YAMNet (finetune)	0.4577	0.943	2.235	3.53M
Joint Model(JM)	0.4609	0.944	2.243	4.51M
JM + Affinity Cross Attention	0.4654	0.944	2.249	4.51M
JM + Bahdanau Cross Attention (LEAN)	0.4677	0.944	2.251	4.58M

TABLE IV
ON-DEVICE PERFORMANCE COMPARISON ON FSD50K DATASET

<i>Model</i>	<i>Model Size</i>	<i>mAP</i>	<i>Inference</i>
LEAN	4.5 MB	0.445	65ms
Att-RNN [35]	-	0.3471	-
MHAtt-RNN [35]	-	0.3317	-
CRNN [35]	-	0.3053	-

B. Temporal feature extractor as Wave Encoder

In the Wave Encoder, we use two Bi-LSTM layers each of 128 units. Context vector yielded from wave encoder is concatenated using equation (1) and trained using objective

function discussed in equations (2) and (3). Using this configuration, we get the mAP of 0.4609. We call this model scheme as Joint Model(JM).

As shown in Table III, our simple joint fusion system (YAMNet + Wave Encoder) surpasses fine-tuned YAMNet in all performance metrics.

C. Variation of cross attention and their impact

Table III captures our various architecture experiments as ablation study. In our first cross attention scheme, we consider an affinity-based attention model for the realignment of temporal embeddings and achieve an mAP of 0.4654. We call this model a "Joint model(JM) with affinity Cross attention". From Table III, we observe that both mAP and dPrime see an improvement compared to fine-tuned YAMNet and Joint model. Another observation is that this scheme of attention does not add any training parameters but still leads to improvement in results which highlights the novelty of this approach. We believe that reason for this improvement due to realignment is the identification of relevant temporal sequences using attentive weights. This mechanism gives higher weights to relevant features and the rest are suppressed by low weighing.

In our next experiment, we adopt Query (Q), Key (K), and Value (V) type structure as discussed in section III-E using Bahdanau [20] additive style of attention. We call this scheme as our proposed LEAN model. Based on our experiments and to keep the model size small, we choose 128 dense units for calculating Q, K, and V. With this setup, we achieve our model best mAP of .4677 and dPrime value of 2.251.

D. Class level performance analysis

Since the baseline system does not contain class-wise results, we compare the class-level performance of our own fine-tuned YAMNet model with the proposed LEAN model.

Table II shows class-wise mAP analysis of different models. We use Baseline(B) and Reference Model (R) nomenclature for convenience where B is compared with R. In the table, the "Overall Improvement" column tells about the number of classes in R that see improvement in mAP over B.

We observe that by introducing a wave encoder, we see an improvement in 70% of the classes compare to YAMNet and further improvement is seen in 48% classes by the realignment scheme.

E. on-Device experiments

Our on-device model results are discussed in Table IV. For deploying on-device, we perform quantization using Tensor-

Flow tflite which results in a model size of nearly 4.5MB. We implement pre-processing steps in the same manner as discussed in section III. We use the Samsung S21 smartphone (Android SDK 30, 12GB RAM, 256GB ROM, Octa-core, Exynos2100 chip) for our on-device experiments. Due to model quantization, we see slight natural drop in mAP from .4677 on GPU to .445 on on-device. We compare our on-device model result with this [35] baseline work which demonstrates several on-device models with best mAP by Att-RNN model as .3471 which is 22% lower than our LEAN model.

V. CONCLUSION

In this paper, we propose a lightweight joint embedding-based audio classification model for on-device deployment called as LEAN which simultaneously extracts temporal and pretrained spatial features from the given audio signal. Further, we use novel temporal features realignment using cross attention which results in improved performance with a slight increase in model parameters. Our proposed LEAN model outperforms the baseline on-device system on the FSD50K dataset with a lesser memory footprint and produced competitive results in comparison to several existing works. We also conduct a detailed analysis of class-level impact using our system for all 200 classes of FSD50K dataset. Our work demonstrates that using a pretrained model, it is possible to achieve further performance improvement using cross-attention-based temporal realignment. In the future work, we aim to replace YAMNet with state of an art pretrained transformer-based feature extractor model and see the performance impact. In addition to this, we wish to leverage label ontology-based mutual relation and dependency knowledge for FSD50K multi-label dataset to improve class-level performance.

REFERENCES

- [1] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2017, pp. 776–780.
- [2] A. Mesaros, T. Heittola, and T. Virtanen, "A multi-device dataset for urban acoustic scene classification," in Workshop on Detection and Classification of Acoustic Scenes and Events (DCASE), 2018, pp. 9–13.
- [3] K. Choi, G. Fazekas, and M. Sandler, "Automatic tagging using deep convolutional neural networks," in Conference of the International Society for Music Information Retrieval (ISMIR), 2016, pp. 805–811.
- [4] Kong, Qiuqiang Cao, Yin Iqbal, Turab Wang, Yuxuan Plumley, Mark. (2019). PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition.
- [5] Prateek Verma and Jonathan Berger, Audio Transformers: Transformer Architectures For Large Scale Audio Understanding, 2021, arXiv, cs.SD.
- [6] Yoon, Seunghyun & Byun, Seokhyun & Jung, Kyomin. (2018). Multimodal Speech Emotion Recognition Using Audio and Text. 10.1109/SLT.2018.8639583.
- [7] <https://github.com/tensorflow/models/tree/master/research/audioset/vggish> : Audioset based Vggish.
- [8] <https://github.com/tensorflow/models/tree/master/research/audioset/YAMNet> : AudioSet based YAMNet.
- [9] Fonseca, Eduardo & Favory, Xavier & Pons, Jordi & Font, Frederic & Serra, Xavier. (2020). FSD50K: an Open Dataset of Human-Labeled Sound Events.
- [10] Gemmeke, Jort & Ellis, Daniel & Freedman, Dylan & Jansen, Aren & Lawrence, Wade & Moore, R. & Plakal, Manoj & Ritter, Marvin. (2017) Audio Set: An ontology and human-labeled dataset for audio events. 776-780. 10.1109/ICASSP.2017.7952261.
- [11] Sarthak, & Shukla, Shikhar & Mittal, Govind. (2019). Spoken Language Identification using ConvNets
- [12] Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," in 22nd ACM International Conference on Multimedia (ACM-MM'14), Orlando, FL, USA, Nov. 2014, pp. 1041–1044.
- [13] Piczak, Karol. (2015). ESC: Dataset for Environmental Sound Classification. 1015-1018. 10.1145/2733373.2806390
- [14] <https://www.kaggle.com/c/freesound-audio-tagging>
- [15] W. Dai, C. Dai, S. Qu, J. Li, and S. Das, "Very deep convolutional neural networks for raw waveforms," in IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2017, pp. 421–425.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [17] K. Choi, G. Fazekas, and M. Sandler, "Automatic tagging using deep convolutional neural networks," in Conference of the International Society for Music Information Retrieval (ISMIR), 2016, pp. 805–811.
- [18] Q. Kong, C. Yu, Y. Xu, T. Iqbal, W. Wang, and M. D. Plumley, "Weakly labelled audioset tagging with attention neural networks," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 27, pp. 1791–1802, 2019.
- [19] Bartz, Christian & Herold, Tom & Yang, Haojin & Meinel, Christoph. (2017). Language Identification Using Deep Convolutional Recurrent Neural Networks
- [20] Bahdanau, Dzmitry & Cho, Kyunghyun & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. ArXiv. 1409.
- [21] Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017). Attention Is All You Need
- [22] Luong, Minh-Thang & Pham, Hieu & Manning, Christopher. (2015). Effective Approaches to Attention-based Neural Machine Translation. 10.18653/v1/D15-1166
- [23] Sandler, Mark & Howard, Andrew & Zhu, Menglong & Zhmoginov, Andrey & Chen, Liang-Chieh. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. 4510-4520. 10.1109/CVPR.2018.00474
- [24] Hochreiter, Sepp & Schmidhuber, Jürgen. (1997). Long Short-term Memory. Neural computation. 9. 1735-80. 10.1162/neco.1997.9.8.1735
- [25] <https://research.google.com/audioset/ontology/index.html>
- [26] Soundfile: <https://pypi.org/project/SoundFile/>
- [27] Tensorflow: <https://www.tensorflow.org/>
- [28] Keras: <https://keras.io>
- [29] <https://github.com/SarthakYadav/fsd50k-pytorch>
- [30] https://www.tensorflow.org/api_docs/python/tf/keras/metrics/AUC
- [31] Koutini, Khaled & Schlüter, Jan & Eghbal-zadeh, Hamid & Widmer, Gerhard. (2021). Efficient Training of Audio Transformers with Patchout.
- [32] Gong, Yuan & Chung, Yu-An & Glass, James. (2021). PSIA: Improving Audio Tagging With Pretraining, Sampling, Labeling, and Aggregation. IEEE/ACM Transactions on Audio, Speech, and Language Processing.
- [33] Niizumi, Daisuke & Takeuchi, Daiki & Ohishi, Yasunori & Harada, Noboru & Kashino, Kunio. (2021). BYOL for Audio: Self-Supervised Learning for General-Purpose Audio Representation. IJCNN52387.2021.9534474.
- [34] Wu, Ho-Hsiang & Seetharaman, Prem & Kumar, Kundan & Bello, Juan. (2022). Wav2CLIP: Learning Robust Audio Representations from Clip.
- [35] <https://doi.org/10.48550/arxiv.2110.14131>, author = Choi, Kwanghee and Kersner, Martin and Morton, Jacob and Chang, Buru, title = Temporal Knowledge Distillation for On-device Audio Classification.
- [36] A. P. Bradley, "The use of the area under the roc curve in the evaluation of machine learning algorithms," Pattern recognition, vol. 30, no. 7, pp. 1145–1159, 1997.